

Chapter 1

Foundations of Adversarial Machine Learning Security

PCC190 – Secure Machine Learning
Prof. Rodrigo C. P. Silva – DECOM/UFOP

October 2, 2026

Contents

Notation	1
1 Why Machine Learning Needs Its Own Security Analysis	1
1.1 A catalogue of vulnerabilities across the pipeline	2
2 Threat Models	2
2.1 Adversary goals	2
2.2 Adversary knowledge	3
2.3 Adversary capabilities	3
3 Attack Surfaces: Training Time versus Inference Time	3
4 Mathematical Formulation of Adversarial Examples	4
4.1 The phenomenon	4
4.2 Two equivalent views	4
4.3 Choice of threat norm	5
4.4 Why do adversarial examples exist? The linearity view	5
4.5 Adversarial risk	5
5 A Taxonomy of Adversarial Attacks	5
6 Overview of Defense Strategies	6
Exercises	7

Notation

Throughout the course we use the following conventions. A classifier is a function $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$ with parameters θ , input space $\mathcal{X} \subseteq \mathbb{R}^d$ and label set $\mathcal{Y} = \{1, \dots, K\}$. For neural networks we write $Z(\mathbf{x}) \in \mathbb{R}^K$ for the logits and $F(\mathbf{x}) = \text{softmax}(Z(\mathbf{x}))$ for the vector of class probabilities, so that $f(\mathbf{x}) = \arg \max_k F_k(\mathbf{x})$. The training loss on a pair $(\mathbf{x}, y)$ is $\mathcal{L}(\theta; \mathbf{x}, y)$, typically the cross-entropy $-\log F_y(\mathbf{x})$. Data are drawn from a distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$. A perturbation is $\delta \in \mathbb{R}^d$, and the perturbed input is $\mathbf{x}' = \mathbf{x} + \delta$. The ℓ_p ball of radius ϵ centered at $\mathbf{x}$ is

$$\mathcal{B}_p(\mathbf{x}, \epsilon) = \{\mathbf{x}' \in \mathcal{X} : \|\mathbf{x}' - \mathbf{x}\|_p \leq \epsilon\}.$$

1 Why Machine Learning Needs Its Own Security Analysis

Classical software security reasons about code: a buffer overflow or an injection exists because some instruction does something the programmer did not intend. Machine learning adds a new kind of artifact whose behavior is not written by anyone. The decision rule of a trained model is induced from data by an optimization procedure, and the only specification the model satisfies is statistical: low expected loss under $\mathcal{D}$. Nothing in that specification constrains behavior on inputs that are rare, or impossible, under $\mathcal{D}$, and nothing constrains behavior when $\mathcal{D}$ itself has been tampered with. This is the root of most of what this course covers.

Three properties make ML systems a distinct security target (Barreno et al., 2006; Huang et al., 2011; Papernot et al., 2018):

1. **Data is part of the program.** Whoever influences the training data influences the learned function. The training set therefore belongs to the trusted computing base, even when it is scraped from the web or contributed by users.
2. **The specification is implicit and average-case.** Accuracy on a test set says nothing about worst-case behavior. An adversary does not sample from $\mathcal{D}$; they search for the inputs where the model is wrong.
3. **Models leak.** A model is a compressed function of its training data. Its outputs, confidences and gradients can reveal information about that data and about the model itself.

1.1 A catalogue of vulnerabilities across the pipeline

It is useful to walk along a typical ML pipeline and ask, at each stage, what can go wrong when someone is actively trying to make it go wrong.

Data collection and labeling. Training data often comes from sources the model owner does not control: crowdsourced labels, user feedback, web crawls, third-party datasets. An attacker who can inject or modify a fraction of these samples can degrade the model or implant hidden behavior. This is *data poisoning* (Biggio et al., 2012), studied in Chapter 3.

Training and the supply chain. Pretrained weights, open-source training code, and outsourced training are all points of trust. A model trained by a third party can contain a *backdoor*: normal behavior on clean inputs, attacker-chosen behavior when a trigger pattern is present (Gu et al., 2019). In federated learning, participants submit updates rather than data, and a malicious participant can poison the global model directly through its update.

Deployment and inference. Once deployed, the model receives inputs from users. An attacker can craft inputs that are classified incorrectly while still looking legitimate to a human or to downstream checks. These are *evasion attacks* and *adversarial examples* (Biggio et al., 2013; Szegedy et al., 2014), the main subject of this chapter and the next.

Model outputs and APIs. A prediction API is an oracle. Repeated queries allow an attacker to infer whether a specific record was in the training set (*membership inference*, Shokri et al., 2017), reconstruct representative inputs or sensitive attributes (*model inversion*, Fredrikson et al., 2015), or train a functional copy of the model (*model extraction*, Tramèr et al., 2016). In collaborative training, shared gradients can even be inverted back to the training inputs (Zhu et al., 2019). These are the topics of Chapter 4.

The three security properties, revisited

It helps to map these vulnerabilities onto the classical CIA triad:

- **Integrity:** the model produces outputs the attacker chose (targeted evasion, backdoors, targeted poisoning).
- **Availability:** the model becomes useless for legitimate users (indiscriminate poisoning, high error rates on clean data).
- **Confidentiality / privacy:** the model reveals information about its training data or about itself (membership inference, inversion, extraction).

2 Threat Models

A security claim without a threat model is meaningless. Saying that a defense “makes a model robust” only has content once we say robust *against whom*, *doing what*, and *knowing what*. The standard way to structure this, which goes back to Barreno et al. (2006) and was refined by Biggio and Roli (2018), is to describe the adversary along three axes: goal, knowledge and capability.

2.1 Adversary goals

Security violation. Which property does the attacker want to break: integrity, availability, or confidentiality (see the box above)?

Specificity. A *targeted* attack aims at a particular outcome, for example forcing an input to be classified as a specific class t , or causing errors only on a specific set of victims. An *untargeted* (or indiscriminate) attack is satisfied with any error.

Formally, for a single input $(\mathbf{x}, y)$, the untargeted goal is $f(\mathbf{x}') \neq y$, while the targeted goal is $f(\mathbf{x}') = t$ for some chosen $t \neq y$. Targeted attacks are strictly harder: every successful targeted attack is also a successful untargeted one, but not the reverse.

2.2 Adversary knowledge

We describe knowledge as the subset of system components the attacker knows: the training data $\mathcal{D}_{\text{train}}$, the feature representation, the architecture of f , the parameters θ , the training algorithm, and any deployed defense. Three settings recur:

- **White-box.** Full knowledge of the model, including parameters and therefore gradients. This is the setting of choice for *evaluating* a defense, because it gives the strongest attacker.
- **Gray-box.** Partial knowledge, for instance the architecture and training data distribution but not the weights, or the model but not the preprocessing defense.
- **Black-box.** Only query access. We further distinguish *score-based* access (the attacker observes $F(\mathbf{x})$ or the top- k scores) from *decision-based* or *label-only* access (the attacker observes only $f(\mathbf{x})$). A *no-box* or transfer setting allows no queries at all to the target.

Kerckhoffs's principle applies

Security should not rely on the secrecy of the model or the defense. If a defense only works because the attacker does not know it exists, it is security through obscurity. Weights leak, architectures are published, and black-box attacks often succeed through transfer from substitute models (Chapter 2). For this reason, defenses must be evaluated against *adaptive* white-box attackers who know the defense (Carlini et al., 2019; Tramèr et al., 2020).

2.3 Adversary capabilities

Capabilities describe what the attacker can *do*, as opposed to what they know.

- **Influence on training versus inference.** Can the attacker modify training data (poisoning), the training procedure (supply chain, federated updates), or only test-time inputs (evasion)?
- **Perturbation budget.** How much can an input be modified? In the image domain this is typically $\|\delta\|_p \leq \epsilon$; in other domains it is a set of allowed transformations (synonym substitutions, adding API calls to malware, rotating a physical object).
- **Fraction of controlled data.** For poisoning, the proportion of training samples the attacker can insert or alter.
- **Query budget.** For black-box attacks, the number of queries allowed before detection or before the cost becomes prohibitive.

Example 2.1 (Writing down a threat model). Consider a content moderation classifier exposed through a public API that returns a binary decision. A reasonable threat model for spammers is: goal is untargeted integrity violation on their own posts (“not spam”); knowledge is black-box, decision-based; capability is to edit their own text with semantics-preserving changes, with a budget of a few hundred queries per post before their account is flagged. Notice how this immediately rules out gradient-based attacks and ℓ_∞ pixel noise as the relevant evaluation, and points instead to decision-based attacks over discrete text transformations.

3 Attack Surfaces: Training Time versus Inference Time

The most important structural distinction in adversarial ML is *when* the attacker acts. The two surfaces are compared in Table 1.

Training-time attacks. The attacker influences the learning process so that the resulting parameters $\hat{\theta}$ serve their goal. Writing the clean training set as S and the attacker’s contribution as S_p , poisoning is naturally a *bilevel* optimization problem:

$$\max_{S_p \in \mathcal{C}} \mathcal{L}_{\text{adv}}(\hat{\theta}(S_p)) \quad \text{s.t.} \quad \hat{\theta}(S_p) \in \arg \min_{\theta} \sum_{(\mathbf{x}, y) \in S \cup S_p} \mathcal{L}(\theta; \mathbf{x}, y), \quad (1)$$

where $\mathcal{C}$ encodes the attacker’s capability (number of points, allowed modifications) and $\mathcal{L}_{\text{adv}}$ encodes the goal (validation error for availability attacks, misclassification of specific targets for integrity attacks). The inner problem is training itself, which makes exact solution expensive; most practical attacks approximate it.

Inference-time attacks. The model is fixed and the attacker manipulates inputs (evasion) or uses outputs as an information channel (privacy and extraction). The canonical evasion problem is a single-level optimization over the input, introduced formally in Section 4.

	Training time	Inference time
Attacker acts on	Training data, labels, model updates, pre-trained weights	Test inputs, query interface
Typical goals	Availability loss, targeted misbehavior, backdoors	Misclassification, information leakage, model theft
Representative attacks	Label flipping, gradient-based poisoning, clean-label poisoning, backdoors	Evasion (adversarial examples), membership inference, inversion, extraction
Persistence	Persistent: affects every later prediction	Per-input or per-query
Optimization structure	Bilevel (Eq. 1)	Single-level over $\mathbf{x}$ or over queries
Covered in	Chapter 3	Chapters 2 and 4

Table 1: Training-time and inference-time attack surfaces.

Remark 3.1. The boundary is not always sharp. Backdoor attacks are planted at training time but *exploited* at inference time by presenting the trigger. In federated learning, gradient inversion happens during training but is an information leak, not a manipulation of the model. Systems with online learning or feedback loops (recommenders, fraud detectors retrained on recent data) blur the distinction further, since today’s inputs are tomorrow’s training data.

4 Mathematical Formulation of Adversarial Examples

4.1 The phenomenon

Szegedy et al. (2014) observed that deep networks with excellent test accuracy could be made to misclassify almost any correctly classified image by adding a perturbation small enough to be invisible. Biggio et al. (2013) had independently formulated test-time evasion as an optimization problem for SVMs and shallow networks. The surprise was not that models make mistakes, but that mistakes are *everywhere*, arbitrarily close to natural data points.

4.2 Two equivalent views

Definition 4.1 (Adversarial example, budget-constrained form). Given a classifier f , a correctly classified input $(\mathbf{x}, y)$, a distance $\|\cdot\|_p$ and a budget $\epsilon > 0$, an *adversarial example* is any $\mathbf{x}' \in \mathcal{B}_p(\mathbf{x}, \epsilon)$ with $f(\mathbf{x}') \neq y$ (untargeted), or $f(\mathbf{x}') = t$ for a chosen $t \neq y$ (targeted).

Since f is a discrete arg max, searching directly for such points is hard. We replace the misclassification condition by a continuous surrogate and solve

$$\mathbf{x}' = \arg \max_{\mathbf{x}' \in \mathcal{B}_p(\mathbf{x}, \epsilon) \cap \mathcal{X}} \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}', y) \quad (\text{untargeted}), \quad (2)$$

$$\mathbf{x}' = \arg \min_{\mathbf{x}' \in \mathcal{B}_p(\mathbf{x}, \epsilon) \cap \mathcal{X}} \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}', t) \quad (\text{targeted}). \quad (3)$$

The constraint $\mathbf{x}' \in \mathcal{X}$ enforces validity of the input, for example $\mathbf{x}' \in [0, 1]^d$ for normalized images.

Definition 4.2 (Minimum-norm form). Alternatively, we can ask for the *smallest* perturbation that changes the decision:

$$\boldsymbol{\delta}^*(\mathbf{x}) = \arg \min_{\boldsymbol{\delta}} \|\boldsymbol{\delta}\|_p \quad \text{s.t.} \quad f(\mathbf{x} + \boldsymbol{\delta}) \neq y, \mathbf{x} + \boldsymbol{\delta} \in \mathcal{X}. \quad (4)$$

The quantity $\rho_p(\mathbf{x}) = \|\boldsymbol{\delta}^*(\mathbf{x})\|_p$ is the ℓ_p distance from $\mathbf{x}$ to the decision boundary, sometimes called the *minimal adversarial perturbation* or *robustness radius* at $\mathbf{x}$.

The two forms answer different questions. The budget-constrained form (Eq. 2) is used for evaluation at a fixed ϵ and for adversarial training. The minimum-norm form (Eq. 4) measures how far the boundary is, and is the basis of attacks such as DeepFool (Moosavi-Dezfooli et al., 2016) and Carlini–Wagner (Carlini and Wagner, 2017). They are linked by $\rho_p(\mathbf{x}) \leq \epsilon \iff$ an adversarial example exists in $\mathcal{B}_p(\mathbf{x}, \epsilon)$.

4.3 Choice of threat norm

The ℓ_p norms used in practice each model a different kind of change:

- ℓ_∞ : $\|\delta\|_\infty = \max_i |\delta_i|$. Every feature may change, but by at most ϵ . For 8-bit images, $\epsilon = 8/255$ is a common benchmark value on CIFAR-10.
- ℓ_2 : Euclidean energy of the perturbation. Allows larger changes on a few features if others are left untouched.
- ℓ_1 and ℓ_0 : sparse perturbations. ℓ_0 counts the number of modified features, modeling attacks that change a few pixels or a few fields of a record.

ℓ_p balls are a proxy, not the goal

The real constraint is semantic: the perturbed input should be “the same thing” to a human or should preserve functionality (a malware sample must still run). ℓ_p balls are mathematically convenient approximations of that set. Many perceptually trivial changes (small rotations, translations, color shifts) have large ℓ_p norm, and some small ℓ_p changes are perceptible. Robustness in one norm does not imply robustness in another. See Gilmer et al. (2018) for a critique of the ℓ_p game as a model of real threats.

4.4 Why do adversarial examples exist? The linearity view

Goodfellow et al. (2015) offered a simple argument. Consider a linear score $s(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ and an ℓ_∞ perturbation of size ϵ . The worst-case perturbation is $\delta = \epsilon \text{sign}(\mathbf{w})$, which changes the score by

$$\mathbf{w}^\top \delta = \epsilon \|\mathbf{w}\|_1.$$

If the entries of $\mathbf{w}$ have average magnitude m , this change is ϵmd : it grows *linearly with the dimension d* while the per-feature change stays fixed at ϵ . In high dimension, many imperceptible changes add up to a large change in the output.

Example 4.1 (Numerical illustration). Take $d = 3072$ (a $32 \times 32 \times 3$ image), $\epsilon = 8/255 \approx 0.031$ and weights with $\|\mathbf{w}\|_1/d = 0.01$. The score shifts by $0.031 \times 0.01 \times 3072 \approx 0.96$, comparable to a typical logit margin, from a perturbation that changes each pixel by at most 8 intensity levels.

Deep networks are piecewise linear (ReLU) or close to linear in large regions, so the same argument applies locally, with $\mathbf{w}$ replaced by the input gradient $\nabla_{\mathbf{x}} \mathcal{L}$. This view directly motivates the Fast Gradient Sign Method of Chapter 2.

The linearity view is not the whole story. Ilyas et al. (2019) argued that adversarial perturbations exploit *non-robust features*: patterns that are genuinely predictive on $\mathcal{D}$ but brittle and meaningless to humans. Under this view, models are not buggy; they are doing exactly what empirical risk minimization rewards. Both perspectives agree on the practical consequence: standard training gives no reason for the decision boundary to be far from the data.

4.5 Adversarial risk

Standard learning minimizes the risk $\mathbb{E}_{(\mathbf{x},y) \sim \mathcal{D}}[\mathcal{L}(\theta; \mathbf{x}, y)]$. The adversarial counterpart replaces the pointwise loss by its worst case over the threat set:

$$R_{\text{adv}}(\theta) = \mathbb{E}_{(\mathbf{x},y) \sim \mathcal{D}} \left[\max_{\mathbf{x}' \in \mathcal{B}_p(\mathbf{x}, \epsilon)} \mathcal{L}(\theta; \mathbf{x}', y) \right]. \quad (5)$$

Minimizing Eq. (5) over θ yields the min-max (saddle point) formulation of Madry et al. (2018), which underlies adversarial training (Chapter 5). The corresponding 0-1 quantity, *robust accuracy* at radius ϵ , is the fraction of test points for which *no* adversarial example exists in $\mathcal{B}_p(\mathbf{x}, \epsilon)$. Note that any attack only gives an *upper bound* on robust accuracy; a lower bound requires certification.

5 A Taxonomy of Adversarial Attacks

Combining the previous sections, an attack can be located along several independent axes. Table 2 summarizes them; Vassilev et al. (2025) provides an up-to-date standardized terminology.

Axis	Values
Phase	Training time (poisoning, backdoors) inference time (evasion, privacy, extraction)
Violation	Integrity availability confidentiality/privacy
Specificity	Targeted untargeted
Knowledge	White-box gray-box black-box (score-based, decision-based) transfer/no-box
Perturbation model	$\ell_\infty, \ell_2, \ell_1, \ell_0$ semantic/functional transformations physical
Scope	Per-input universal (one perturbation for many inputs)
Algorithm family	Gradient-based optimization-based gradient estimation random search boundary walking transfer

Table 2: Axes of the attack taxonomy.

Evasion attacks by knowledge. Since Chapter 2 is organized around this axis, it is worth previewing it. In the *white-box* setting the attacker solves Eq. (2) or Eq. (4) with exact gradients (FGSM, PGD, C&W). In the *score-based* setting gradients are unavailable but can be estimated from output scores by finite differences or replaced by random search (ZOO, NES, Square Attack). In the *decision-based* setting only the label is observed, and attacks walk along the decision boundary (Boundary Attack, HopSkipJump). Finally, *transfer attacks* craft examples on a surrogate model and rely on the empirical fact that they often fool the target too.

Universal and physical attacks. Some attacks seek a single perturbation that fools the model on most inputs, or perturbations that survive printing, camera capture and changes in viewpoint (Kurakin et al., 2017; Athalye et al., 2018b). These relax the per-input assumption and tighten the realism constraints; they are discussed in Chapter 7.

6 Overview of Defense Strategies

Defenses can be grouped by *what they change* and *what guarantee they offer*. This section is a map; Chapter 5 develops the main families and Chapter 6 the evaluation methodology.

Robust training. Change the learning objective so the model is trained against the threat. *Adversarial training* approximately minimizes Eq. (5) by generating adversarial examples during training (Goodfellow et al., 2015; Madry et al., 2018); variants such as TRADES make the accuracy-robustness trade-off explicit (Zhang et al., 2019). This is the most reliable empirical defense to date, at substantial computational cost and some loss of clean accuracy.

Certified defenses. Provide a mathematical guarantee that $f(\mathbf{x}') = f(\mathbf{x})$ for all $\mathbf{x}' \in \mathcal{B}_p(\mathbf{x}, r)$, for some certified radius r . Approaches include convex relaxations and bound propagation (Wong and Kolter, 2018) and randomized smoothing (Cohen et al., 2019). Certificates are valid against *any* attack within the threat model, but they are typically available only for small radii and come with accuracy and computational costs.

Input transformation and preprocessing. Remove or disrupt perturbations before classification (JPEG compression, denoising, quantization, random resizing). Cheap and model-agnostic, but historically easy to break once the attacker accounts for the transformation.

Detection. Train a separate component to flag adversarial inputs and reject them. Detectors are themselves models and can be attacked jointly with the classifier; most published detectors have been bypassed by adaptive attacks.

Data-centric defenses against poisoning. Sanitize training data, use robust statistics or robust aggregation (for federated learning), and inspect models for backdoors.

Privacy defenses. Differential privacy, output perturbation and restricting the information returned by APIs address the confidentiality attacks of Chapter 4.

Gradient masking and the false sense of security

Many proposed defenses appear robust because they make gradients uninformative (shattered, stochastic, or vanishing gradients), not because the model is correct on adversarial inputs. Defensive distillation

(Papernot et al., 2016) is a classic example that was later broken (Carlini and Wagner, 2017). Athalye et al. (2018a) showed that most defenses at a major venue relied on such *obfuscated gradients* and could be circumvented with techniques such as BPDA and Expectation over Transformation. Warning signs include: black-box attacks outperforming white-box ones, single-step attacks outperforming iterative ones, and robust accuracy not reaching zero as ϵ grows (Carlini et al., 2019).

Principles for the rest of the course

1. State the threat model explicitly before claiming any security property.
2. Evaluate against the strongest attacker consistent with the threat model, which includes knowledge of the defense.
3. An attack yields an upper bound on robust accuracy; only certification yields a lower bound.
4. Robustness to one perturbation model does not carry over to others.

Exercises

1. Write a complete threat model (goal, knowledge, capabilities) for: (a) an attacker evading a malware detector deployed on endpoints; (b) a participant in a federated learning consortium of hospitals who wants the global model to misdiagnose a specific rare condition.
2. Show that for a linear binary classifier $f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$, the minimal ℓ_2 adversarial perturbation at $\mathbf{x}$ is $\delta^* = -\frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|_2} \mathbf{w}$ (up to an arbitrarily small overshoot). What is the minimal ℓ_∞ perturbation? (Hint: use dual norms.)
3. Explain why every successful targeted attack yields a successful untargeted attack, and give an example where untargeted attacks succeed on all inputs but a targeted attack toward a fixed class t fails on most of them.
4. A paper reports that its defense achieves 85% accuracy under white-box PGD but only 70% under a transfer attack from an undefended model. What does this suggest, and what experiments would you request as a reviewer?

References

- Athalye, A., Carlini, N., and Wagner, D. (2018a). Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 274–283.
- Athalye, A., Engstrom, L., Ilyas, A., and Kwok, K. (2018b). Synthesizing robust adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 284–293.
- Barreno, M., Nelson, B., Sears, R., Joseph, A. D., and Tygar, J. D. (2006). Can machine learning be secure? In *Proceedings of the ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, pages 16–25.
- Biggio, B., Corona, I., Maiorca, D., Nelson, B., Šrndić, N., Laskov, P., Giacinto, G., and Roli, F. (2013). Evasion attacks against machine learning at test time. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, pages 387–402.
- Biggio, B., Nelson, B., and Laskov, P. (2012). Poisoning attacks against support vector machines. In *Proceedings of the 29th International Conference on Machine Learning (ICML)*.
- Biggio, B. and Roli, F. (2018). Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition*, 84:317–331.
- Carlini, N., Athalye, A., Papernot, N., Brendel, W., Rauber, J., Tsipras, D., Goodfellow, I., Madry, A., and Kurakin, A. (2019). On evaluating adversarial robustness. *arXiv preprint arXiv:1902.06705*.
- Carlini, N. and Wagner, D. (2017). Towards evaluating the robustness of neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, pages 39–57.
- Cohen, J., Rosenfeld, E., and Kolter, J. Z. (2019). Certified adversarial robustness via randomized smoothing. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 1310–1320.

- Fredrikson, M., Jha, S., and Ristenpart, T. (2015). Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1322–1333.
- Gilmer, J., Adams, R. P., Goodfellow, I., Andersen, D., and Dahl, G. E. (2018). Motivating the rules of the game for adversarial example research. *arXiv preprint arXiv:1807.06732*.
- Goodfellow, I. J., Shlens, J., and Szegedy, C. (2015). Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*.
- Gu, T., Liu, K., Dolan-Gavitt, B., and Garg, S. (2019). BadNets: Evaluating backdooring attacks on deep neural networks. *IEEE Access*, 7:47230–47244.
- Huang, L., Joseph, A. D., Nelson, B., Rubinstein, B. I. P., and Tygar, J. D. (2011). Adversarial machine learning. In *Proceedings of the 4th ACM Workshop on Security and Artificial Intelligence (AISec)*, pages 43–58.
- Ilyas, A., Santurkar, S., Tsipras, D., Engstrom, L., Tran, B., and Madry, A. (2019). Adversarial examples are not bugs, they are features. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Kurakin, A., Goodfellow, I., and Bengio, S. (2017). Adversarial examples in the physical world. In *ICLR Workshop Track*.
- Madry, A., Makelov, A., Schmidt, L., Tsipras, D., and Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*.
- Moosavi-Dezfooli, S.-M., Fawzi, A., and Frossard, P. (2016). DeepFool: A simple and accurate method to fool deep neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2574–2582.
- Papernot, N., McDaniel, P., Sinha, A., and Wellman, M. P. (2018). SoK: Security and privacy in machine learning. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 399–414.
- Papernot, N., McDaniel, P., Wu, X., Jha, S., and Swami, A. (2016). Distillation as a defense to adversarial perturbations against deep neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, pages 582–597.
- Shokri, R., Stronati, M., Song, C., and Shmatikov, V. (2017). Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy (S&P)*, pages 3–18.
- Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., and Fergus, R. (2014). Intriguing properties of neural networks. In *International Conference on Learning Representations (ICLR)*.
- Tramèr, F., Carlini, N., Brendel, W., and Madry, A. (2020). On adaptive attacks to adversarial example defenses. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Tramèr, F., Zhang, F., Juels, A., Reiter, M. K., and Ristenpart, T. (2016). Stealing machine learning models via prediction APIs. In *25th USENIX Security Symposium*, pages 601–618.
- Vassilev, A., Oprea, A., Fordyce, A., Anderson, H., Davies, X., and Hamin, M. (2025). Adversarial machine learning: A taxonomy and terminology of attacks and mitigations. Technical Report NIST AI 100-2e2025, National Institute of Standards and Technology.
- Wong, E. and Kolter, J. Z. (2018). Provable defenses against adversarial examples via the convex outer adversarial polytope. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 5286–5295.
- Zhang, H., Yu, Y., Jiao, J., Xing, E., El Ghaoui, L., and Jordan, M. (2019). Theoretically principled trade-off between robustness and accuracy. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 7472–7482.
- Zhu, L., Liu, Z., and Han, S. (2019). Deep leakage from gradients. In *Advances in Neural Information Processing Systems (NeurIPS)*.